3.Build an Artificial Neural Network by implementing the Back propagation Algorithm and test the same using appropriate data sets.

```
import numpy as np
inputNeurons=2
hiddenlayerNeurons=4
outputNeurons=2
iteration=6000

input  = np.random.randint(1,5,inputNeurons)
output = np.array([1.0,0.0])
hidden_layer=np.random.rand(1,hiddenlayerNeurons)
hidden_biass=np.random.rand(1,hiddenlayerNeurons)
output_bias=np.random.rand(1,outputNeurons)

hidden_weights=np.random.rand(inputNeurons,hiddenlayerNeurons)
output_weights=np.random.rand(hiddenlayerNeurons,outputNeurons)
def sigmoid (layer):
   return 1/(1 + np.exp(-layer))

def gradient(layer):
   return layer*(1-layer)

for i in range(iteration):
   hidden_layer=np.dot(input,hidden_weights)
   hidden_layer=sigmoid(hidden_layer+hidden_biass)

   output_layer=np.dot(hidden_layer,output_weights)
   output_layer=sigmoid(output_layer+output_bias)
```

```
    error = (output-output_layer)
    gradient_outputLayer=gradient(output_layer)
    error_terms_output=gradient_outputLayer * error

error_terms_hidden=gradient(hidden_layer)*np.dot(error_terms_output,output_weights.T)
    gradient_hidden_weights =
np.dot(input.reshape(inputNeurons,1),error_terms_hidden.reshape(1,hiddenlayerNeurons))
    gradient_ouput_weights =
np.dot(hidden_layer.reshape(hiddenlayerNeurons,1),error_terms_output.reshape(1,output
Neurons))


    hidden_weights = hidden_weights + 0.05*gradient_hidden_weights
    output_weights = output_weights + 0.05*gradient_ouput_weights
    if i<50 or i>iteration-50:
      print("**********************")
      print("iteration:",i,"::::",error)
      print("###utput####o####",output_layer)
```